

P2P Paradigm

Lenuta Alboaie (lenuta.alboaie@info.uaic.ro)

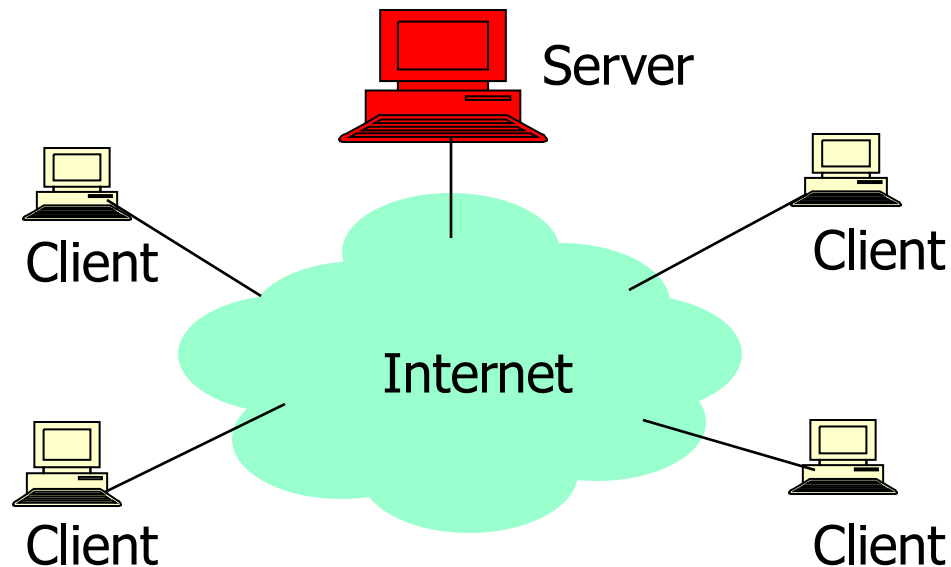
Andrei Panu (andrei.panu@info.uaic.ro)

Content

- **Peer-to-peer (P2P) paradigm**
 - Preliminaries
 - Definitions
 - Characteristics
 - Application types
 - Infrastructures
 - Instruments

Preliminaries

...let's remember the **client/server model**



Preliminaries

...let's remember the **client/server model**

- Usually, we look at the client as at a component having low computational capabilities
- The server is maintained and managed centrally

Problems of client/server architecture:

- The lack of robustness
- Lack of safety (reliability)
- Lack of scalability
- Vulnerability to attacks

Definitions

Peer = *one that is of equal standing with another*
(according to Webster)

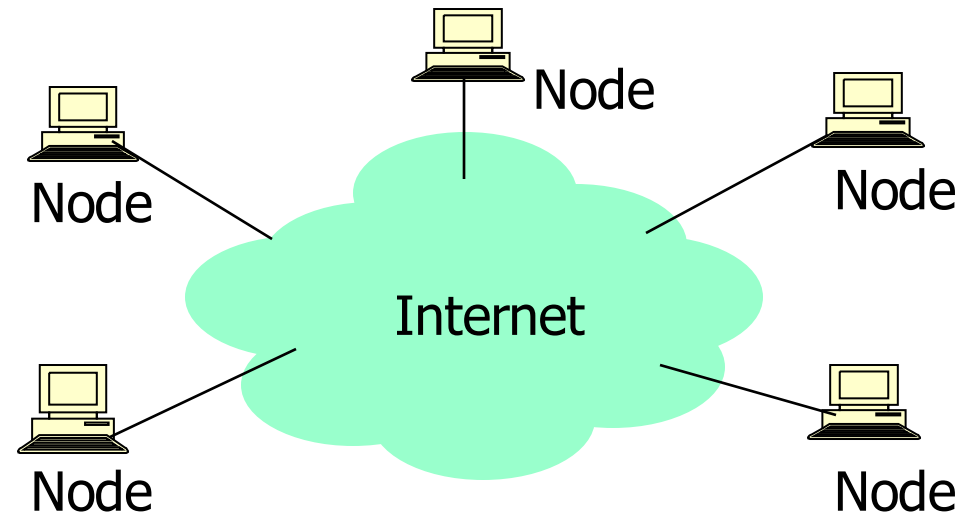
Peer-to-peer (P2P) = network architecture where nodes are relatively equal

- Meaning that each node is capable of performing specific network functions

Definitions

P2P Systems, in the strict sense, are fully distributed systems

- All nodes are fully equivalent in terms of functionality and the activities that they can perform



Obs.: **Pure P2P systems** are rare (e.g., Gnutella), most are hybrids, having super-nodes or servers with different roles (data search, control)

Definitions

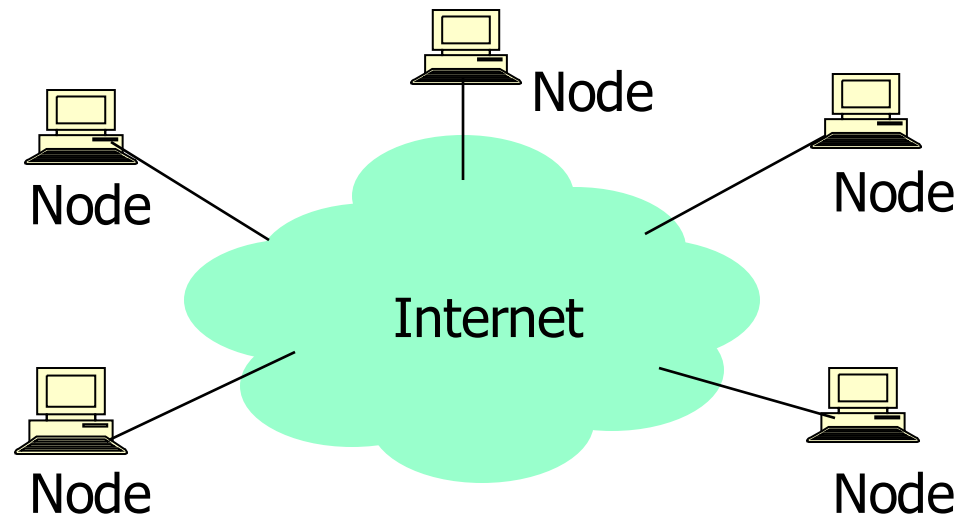
- Nodes

- can consume and provide data
- any node can initiate a connection

- There is no centralized data source =>

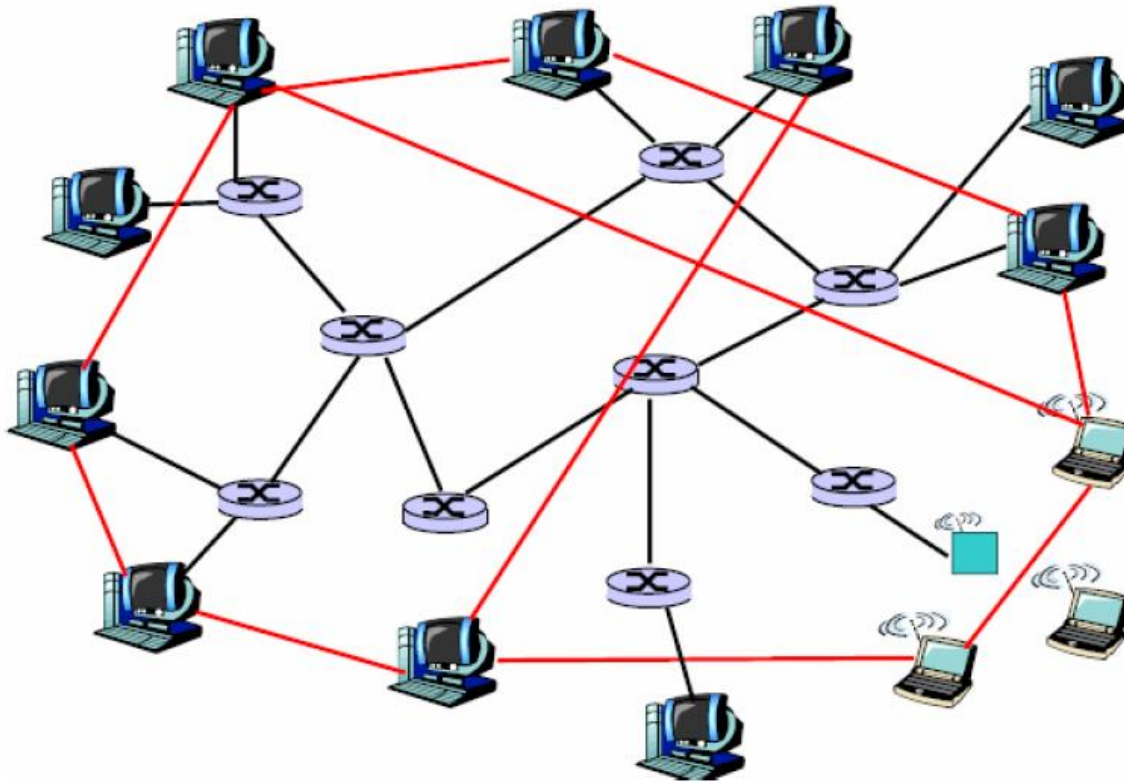
A form of democracy
on the Internet

Copy-right
protection threatened



Definitions

- “**P2P** is the class of applications that rely on the resources (storage, processing, content, human presence) available at the edges of the Internet



*Edges of the Internet
(overlay networks)*

Definitions

“**P2P** is a class of applications that take advantage of resources – storage, cycles, content, human presence – available at the edges of the Internet. Because accessing these decentralized resources means operating in an environment of unstable and unpredictable IP addresses, P2P nodes must operate outside the DNS system and have significant, or total autonomy from central servers”

“A distributed network architecture may be called a **P2P** network if the participants share a part of their own resources. These shared resources are necessary to provide the service offered by the network. The participants of such a network are both resource providers and resource consumers”

Characterization

Characteristics:

- Sharing computational resources through interchange and less directly through mediation offered by a centralized authority (server)
 - Centralized servers can be used to perform specific activities (e.g., P2P network initialization, adding new nodes in the network, etc.)
 - Ideally, nodes participate actively in accomplishing operations such as location & caching nodes / content, routing information, transferred resource management, etc.

Characterization

Characteristics:

- The ability to address instability and variations of network connectivity, with adaptation for the occurred errors or for the node dynamics
 - P2P network topology is adaptive and fault tolerant; nodes are self-organized in order to maintain connectivity and network performance

Characterization

P2P network is an overlay over the physical network

- It is at Application Level => flexibility
- Virtual edges are TCP connections or pointers to IP addresses
- P2P network maintenance is done by periodically verifying connectivity (ping) or existence (messages "still alive?")
- When a node fails, the P2P system could set new edges
- Nodes proximity (physical) is not important
- P2P network can be structured or not

Aims and benefits

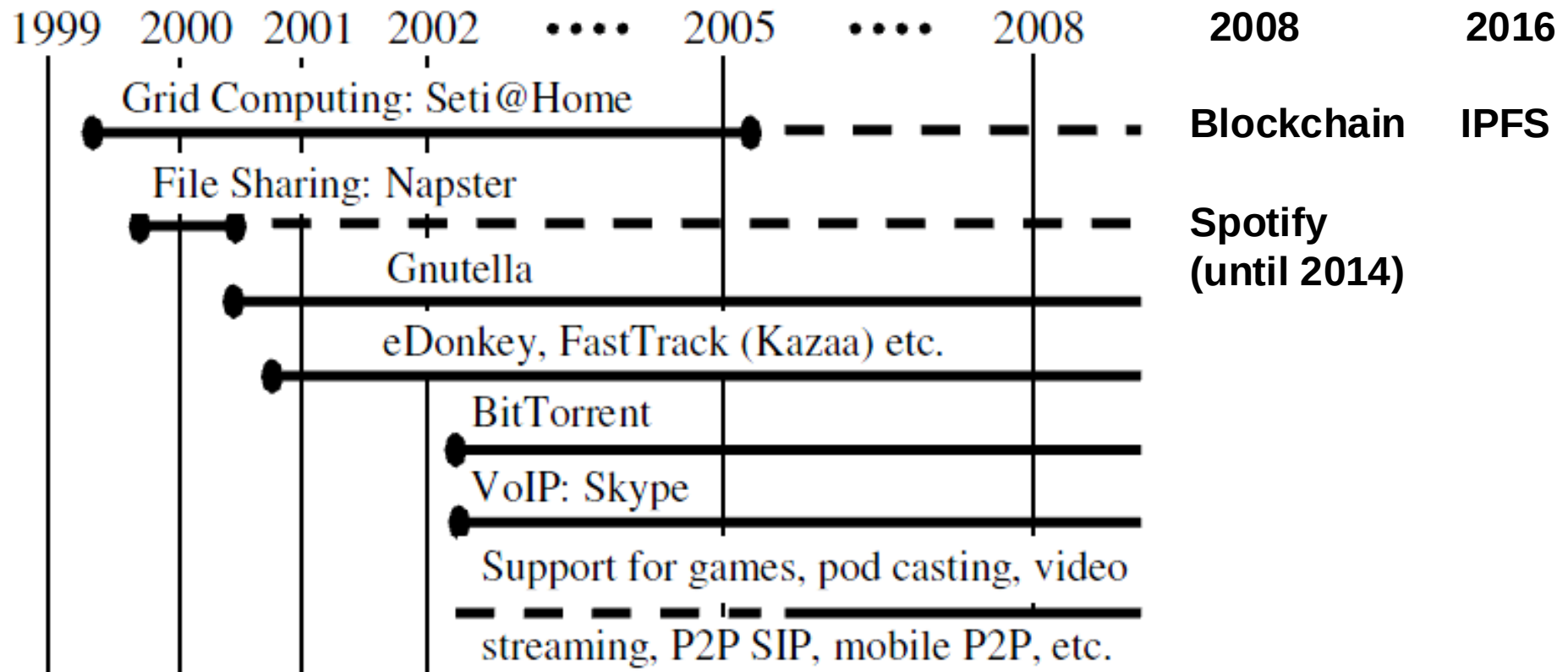
- **Efficient use of resources**
 - Unused bandwidth, storage resources, processing power available to the network edges
- **Scalability**
 - Without centralized information, without bottlenecks (communication and computing)
 - Integration of resources is done naturally during the system use
- **Reliability**
 - The existence of copies of data
 - Geographical distribution
 - No more "single point of failure"
- **Easy administration**
 - The nodes are self-organised
 - Increased fault tolerance and load balancing
 - Increased autonomy
- **Anonymity**
 - Hard to accomplish in a centralized environment
- **Dynamism**
 - Dynamic environment
 - Ad-hoc collaboration and communication

Disadvantages / Problems

- P2P architectures are probabilistic
 - Unpredictable resource location
 - Resources are volatile
- Absence of centralized control
 - Issues with imposing an authority on applications, content and users
 - Difficulties in detecting and identifying users (anti-social aspects)
- Encouraging the use of P2P systems for abusive and illegal purposes (e.g., copyright of digital content)
 - Lack of trust at the commercial/business level
 - Security issues (future course)

Evolution...

Timeline of Popular Peer-to-Peer Protocols



Types of Applications

- **Communication & collaboration**

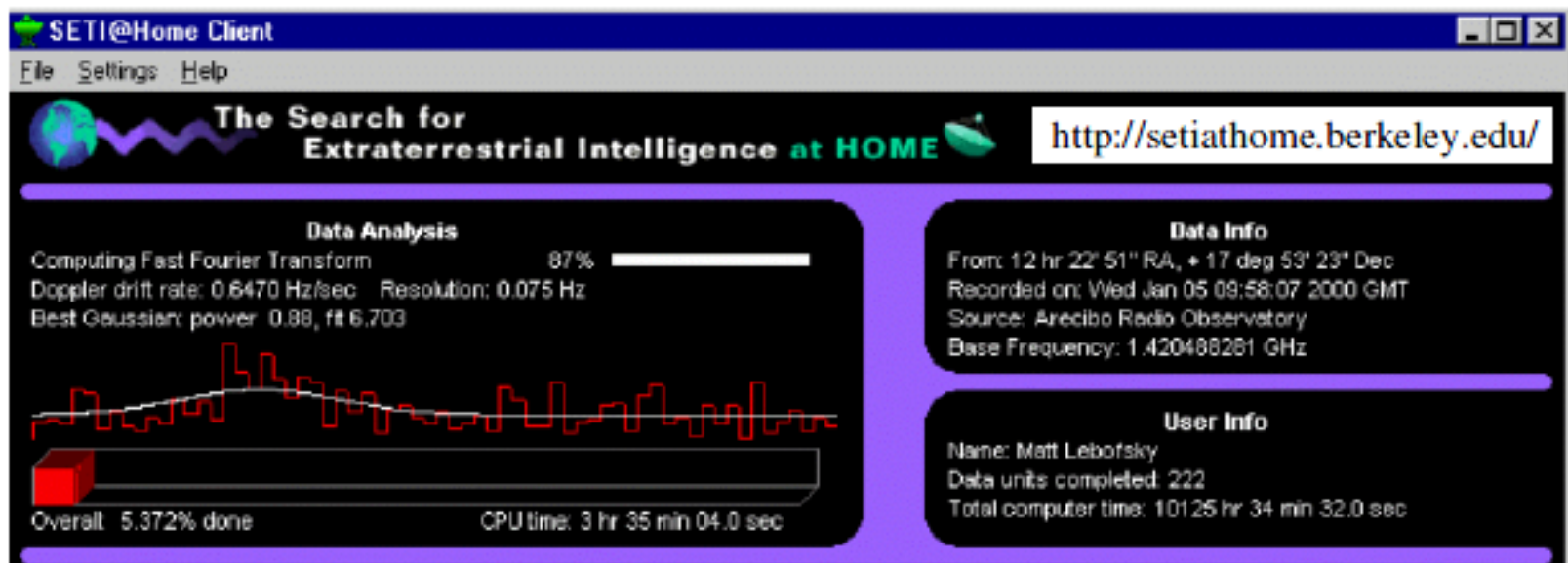
- Systems that provide an infrastructure to facilitate direct communication & collaboration, often in real time between nodes
- Conversational systems (chat, instant messaging):
IRC (Internet Relay Chat), ICQ (1996), YM !, MSN Messenger, Skype, P2P multicast systems (e.g., Cirrus - Adobe Flash <http://labs.adobe.com/technologies/cirrus/>; WebRTC) ...

- **Distributed Computing**

- Systems that use the computational resources of available nodes (processor cycles)
- Solving problems through *divide-et-impera*: SETI@home (*Search for Extra-Terrestrial Intelligence*-Berkeley), genome@home
 - P2P network is a kind of a computational Grid (... master course)

Types of Applications | Distributed Computing - Example

SETI@Home: A Public-Resource Computing Experiment



- ❑ Radio telescope signal analysis has insatiable appetite for computing power
- ❑ Usage of computers in homes and offices around the world has provided unprecedented computing power
- ❑ Grid computing application via peer-to-peer approach under central control

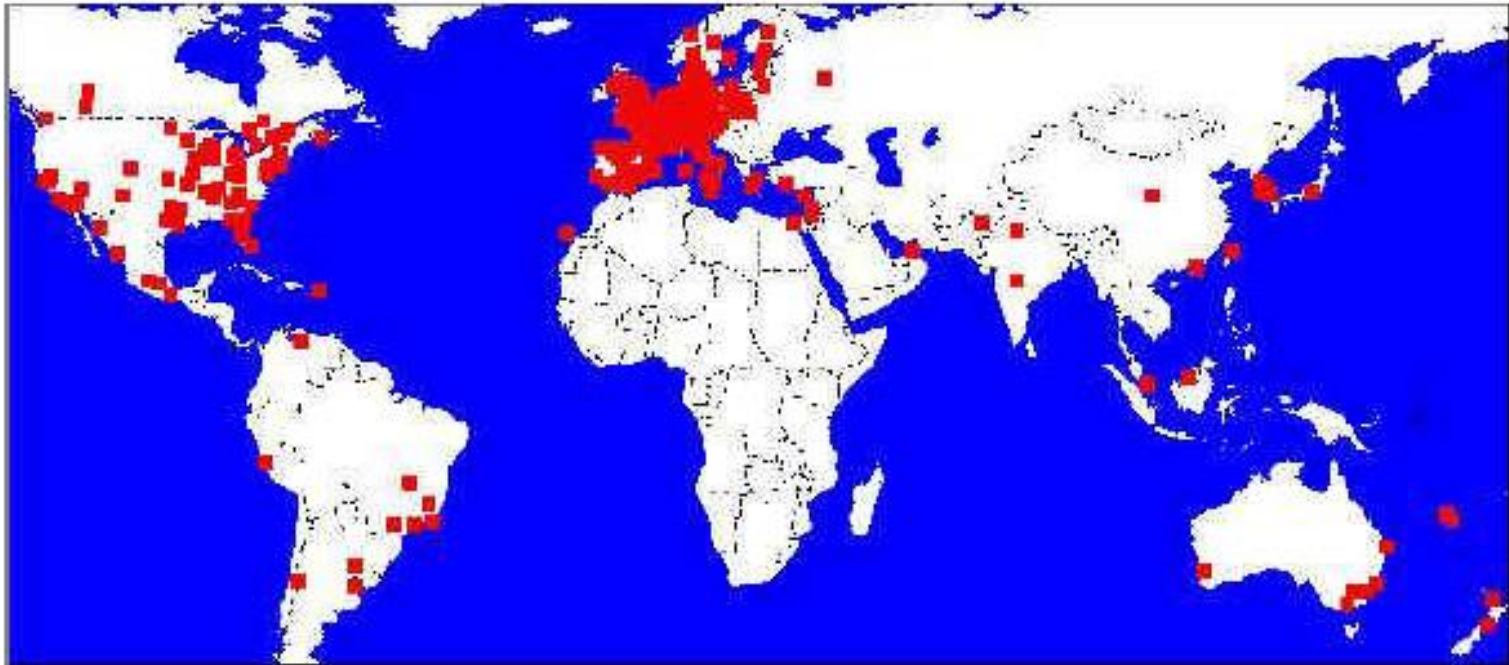
Types of Applications

- **Storage systems (databases)**
 - Design of distributed database infrastructure based on P2P
 - PIER – a scalable engine for distributed query (<http://pier.cs.berkeley.edu/>)
 - Edutella – open-source project for queries and *meta-data* storage (P2P for Semantic Web)
- **Distribution of digital content**
 - Systems & Infrastructure for sharing digital resources (multimedia and other data) among users
 - File sharing applications (e.g., Napster, Gnutella, KaZaA, Freenet, BitTorrent, eDonkey, etc.)
 - Distributed storage media for publishing, organizing, indexing, searching and retrieving data in secure & efficient manner (e.g., PAST, Chord, Groove, Mnemosyne, Avalanche, etc.)

Types of Applications

- **Distribution of digital content | Example**

P2P File-Sharing: Fast distribution of large files



Example: Harry Potter III early propagation after 2 hours on May 28th 2004 (Source: www.itic.ca/DIC/News/archive.html)

Types of Applications

- **Distribution of content through P2P**
 - P2P systems for “interchanging files”
 - In some, nodes transfer files one at a time
 - It offers facilities for realizing a P2P network and for searching & transferring files between nodes
 - Security, availability and persistence are not supported
 - Examples: Napster, KaZaA, Gnutella, BitTorrent

Types of Applications

- **Distribution of content through P2P**
 - P2P systems for publishing & storing content
Users can publish, store and distribute digital content, based on access rights (privileges)
 - Focus is on security and persistence
 - Some systems offer facilities regarding collaboration between users
 - Example: Scan, Groove, Freenet, MojoNation, Tangler

Types of Applications

- **Distribution of content through P2P**
 - Infrastructure for:
 - **Routing & Localization:**
Chord, Can, Pastry, Tapestry, Kademlia
 - **Anonymity:**
Onion Routing, ZeroKnowledge, Freedom, Tarzan
 - **Reputation management:**
EigenTrust, PeerTrust

Infrastruct.(Routing & Localization)

- Localization and routing mechanisms that can be adopted depend on:
 - Topology
 - Structure
 - The degree of centralization of the *overlay network*

Infrastruct.(Routing & Localization)

- **Aspects regarding centralization**
 - **Pure decentralized architectures**: all nodes perform exactly the same activities, by playing roles of servers and clients simultaneously, without the benefit of a central coordination
 - Nodes are called **servents (SERVers + clieENTS)**

Infrastruct.(Routing & Localization)

- **Aspects regarding centralization**
 - **Partially centralized architectures**: some nodes have a more important role (e.g., storing local indexes for shared folders)
 - Nodes become **super-nodes** in accordance with the policies of each P2P system
 - Super-node role is determined dynamically
 - **Hybrid decentralized architectures**: there is a central server enabling the interaction between nodes, keeping catalogs with metadata of files
 - Servers can identify and verify the storage nodes
 - The systems are called *broker mediated*

Infrastruct.(Routing & Localization)

- **Aspects regarding network structure:**
 - **Unstructured:** placing content is completely independent of overlay network topology
 - The content must be located
 - Search strategies by "brute force": flooding the network - requests propagated via BFS / DFS
 - More sophisticated strategies: random path, probabilistic methods, etc.
 - **Loosely structured:** although the content location is not completely specified, it is affected by routing
 - Category located between structured and unstructured networks

Infrastruct.(Routing & Localization)

- **Aspects regarding network structure:**
 - **Structured:** topology is controlled, files are placed in precise locations
 - A mapping between the content (file identifier) and the location (node address) is performed
 - Like a distributed routing table
 - *exact-match queries* can be performed in a scalable way
 - The structure used to guide message routing is difficult to maintain for transient nodes (with high rates of attachment and disconnection from the network)

Infrastruct.(Routing & Localization)

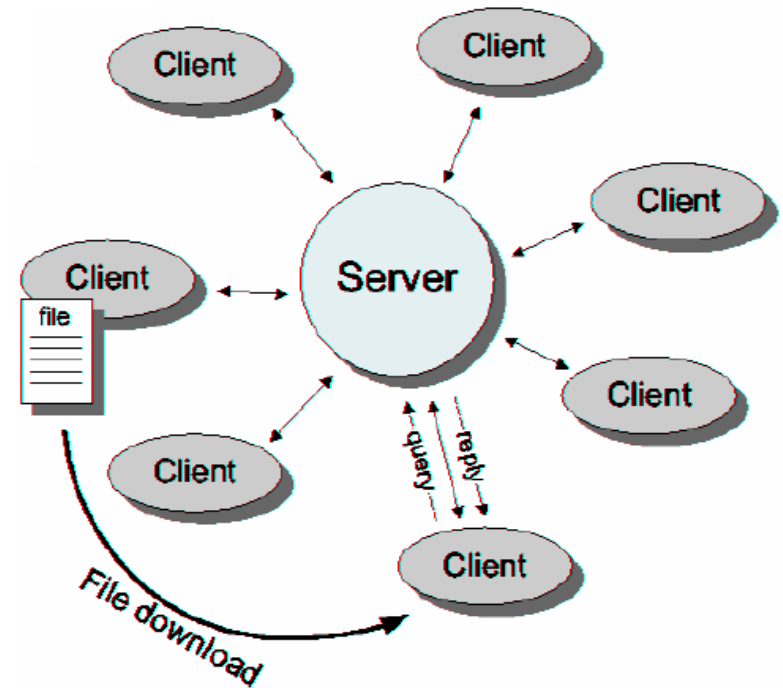
	Centralization		
	Hybrid	Partial	Absent
Unstructured	Napster Publius	BitTorrent KaZaA Morpheus Edutella	Gnutella FreeHaven
Structured Infrastructure			Chord, CAN, Tapestry, Pastry
Structured Systems			OceanStore Scan, PAST, Kademlia, Tarzan

Unstructured Architectures

Hybrid Decentralized

- Each client computer stores shared content (files)
- The central server keeps a table with registered users (IP, bandwidth, ...) + a table with the list of files for user & metadata

Example: **Napster, Publius**



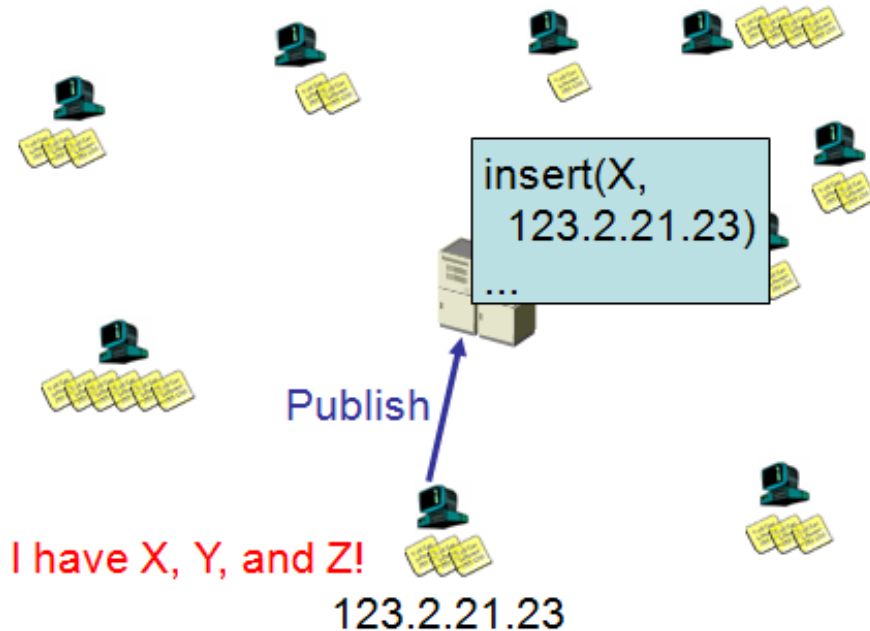
Unstructured Architectures

Napster

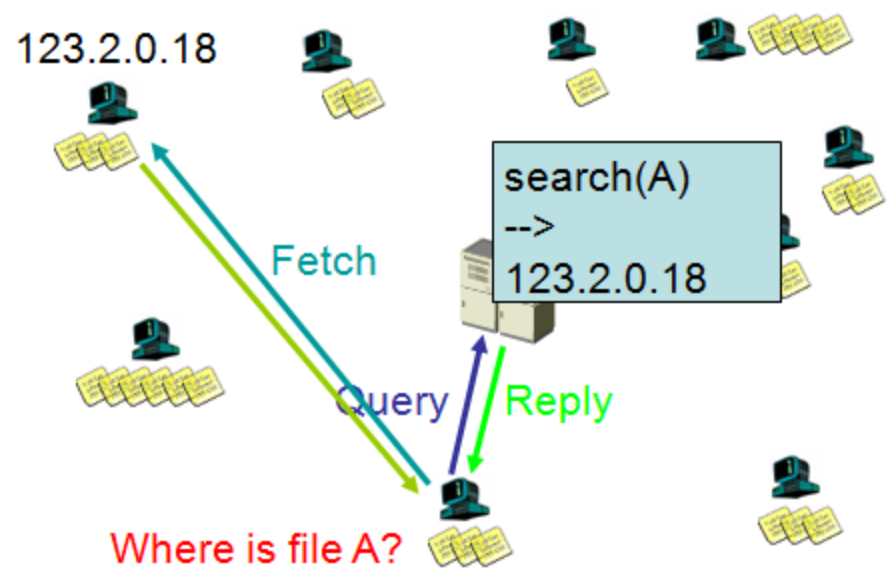
- 1999: Sean Fanning releases Napster
- It reached 1.5 million concurrent users
- Centralized database - operations:
 - **Join**: the client contacts the central server (via TCP)
 - **Publish**: report a list of files to the central server
 - **Search**: query server => it returns a peer that stores the requested file
 - **Fetch**: take the file directly from peer (the one with the best transfer rate)
- July 2001: Napster was closed

Unstructured Architectures

Napster: Publish



Napster: Search



Discussions:

The server does all the processing

We have "single point of failure"

Scalability problems, some systems do not allow adding other servers (the list of available servers is static)

Unstructured Architectures

Pure Decentralized

- An overlay network is built using routing mechanisms based on IP
- There is no central coordination
- Users are connecting via an application that has a dual role – *servent*
- Communication between *servents* is based on a protocol at the application level, with 4 types of messages:
 - Ping – requires a node to announce himself
 - Pong – reply to the *ping* message (IP, port, number and file sizes)
 - Query – search request (search string + minimum speed of transfer)
 - Query hints – response (IP, port, speed, file length, file index)

Unstructured Architectures

Pure Decentralized

- Searching is done by flooding
 - If you don't have the desired file, ask n neighbors
 - If they do not have the file, they will ask their neighbors in maximum m hops
 - On the way back, the answers will be returned (not the files' content)
- Each message has a TTL attached
- Example: **Gnutella**

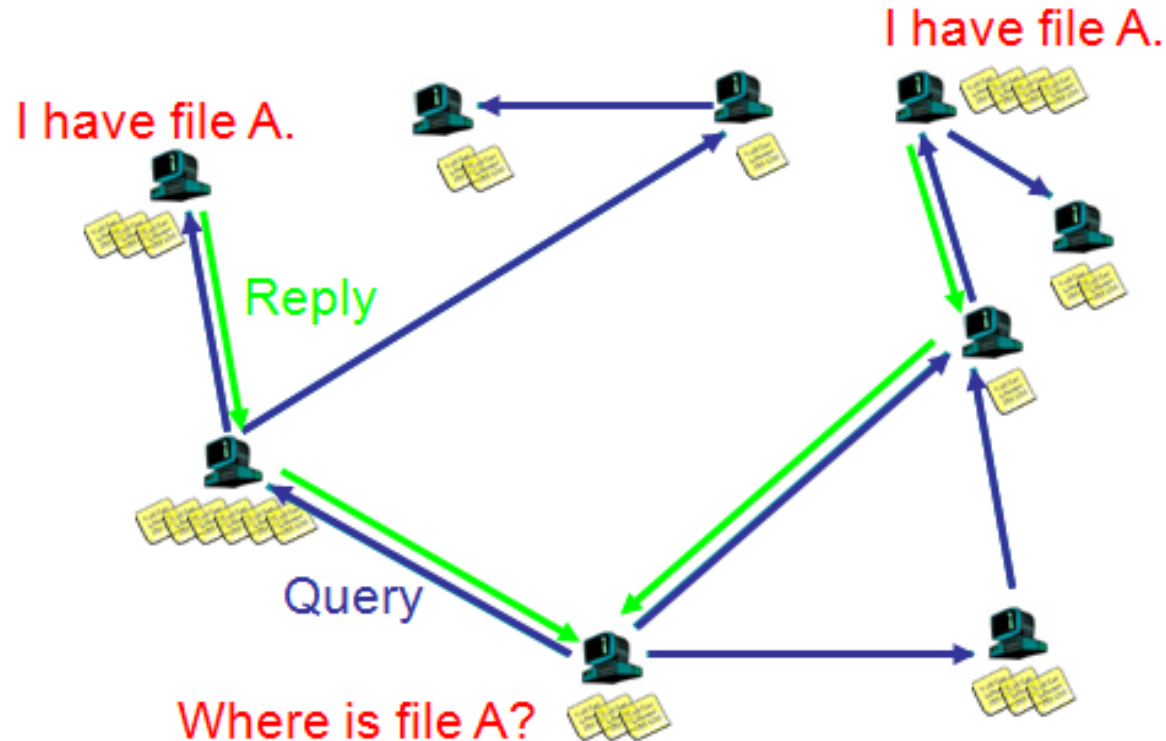
Unstructured Architectures

Gnutella

- 2000: L. Frankel and T. Pepper (Nullsoft) launches Gnutella
- Client applications appear: Bearshare, Morpheus, LimeWire
- *Query Flooding*:
 - **Join**: when joining, the client contacts a few nodes that became his neighbors
 - **Publish**: Not required
 - **Search**: asks neighbors, who ask their neighbors ...
 - There is a TTL limiting the spread
 - **Fetch**: take the file directly from peer

Unstructured Architectures

Gnutella



Issues:

- Search time is... $O(?)$
- The nodes leave often => unstable network

Unstructured Architectures

Partially centralized

- Use the concept of **super-node**: that performs various activities in a P2P network (e.g., indexing, caching)
- The nodes are automatically chosen as super-nodes if they have enough bandwidth and computational power
- All requests are sent initially to super-nodes
- Advantages: resource discovery time is less
- Example: **KaZaA**

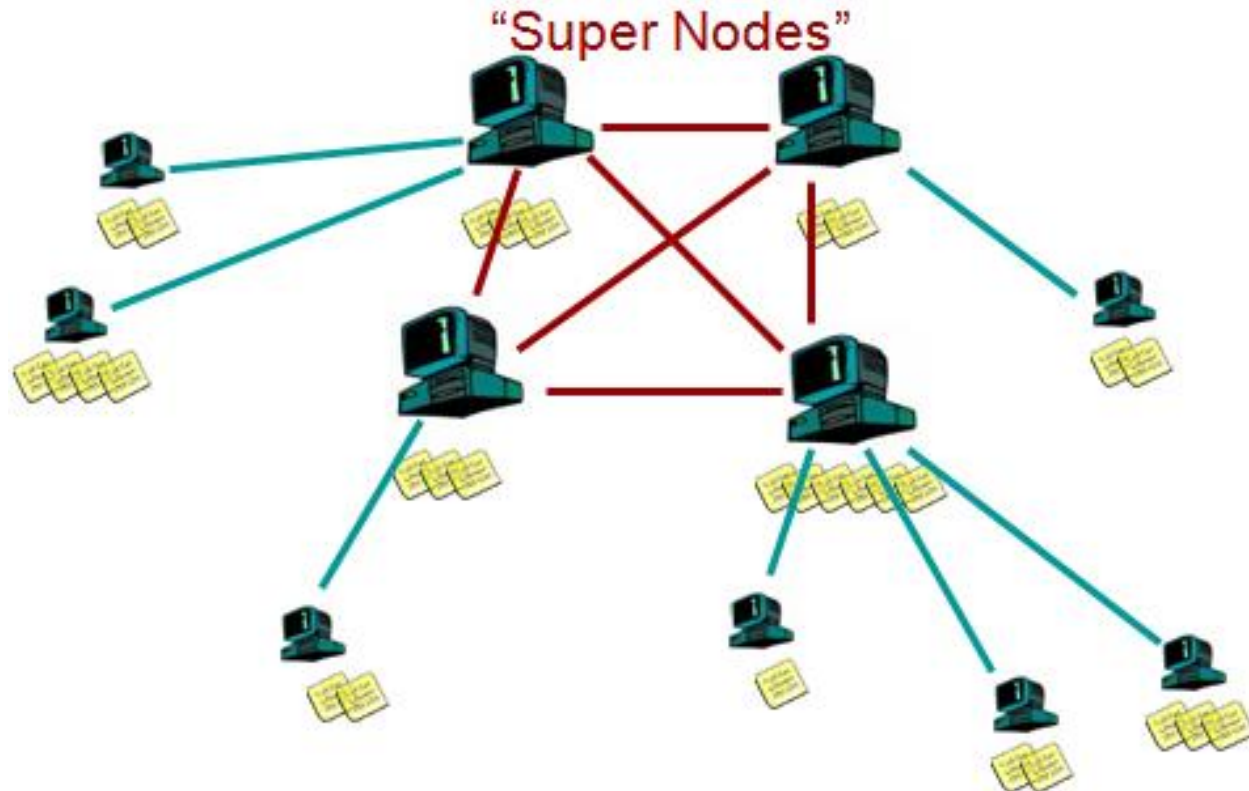
Unstructured Architectures

KaZaA

- 2001: KaZaA is launched
- Client applications appear: Morpheus, giFT
- It utilizes a mechanism of “*smart*” *query flooding*:
 - **Join**: when joining, the client contacts a *super-node* (it can become super-node later)
 - **Publish**: sends the list of files to the *super-node*
 - **Search**: send a query to the *super-node*, and *super-nodes* interrogate each other
 - **Fetch**: take the file directly from the *peer(s)*; the file can be obtained simultaneously from multiple peers

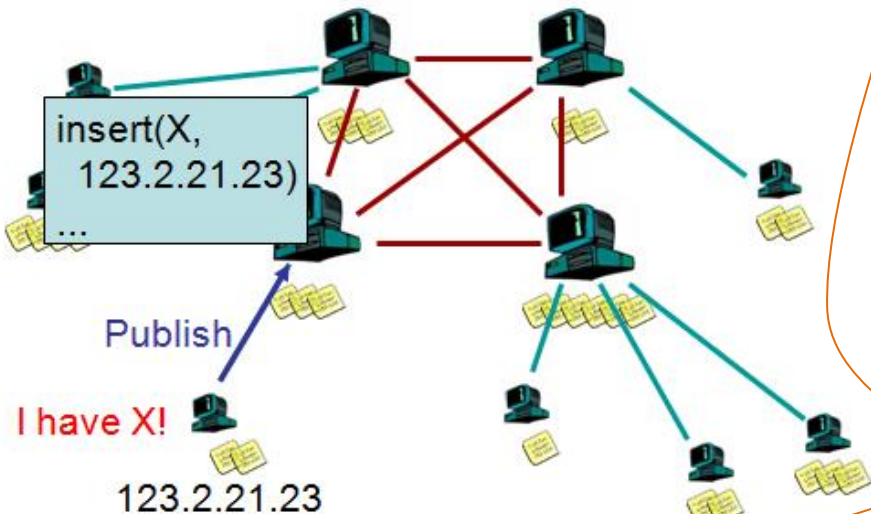
Unstructured Architectures

KaZaA: Network design

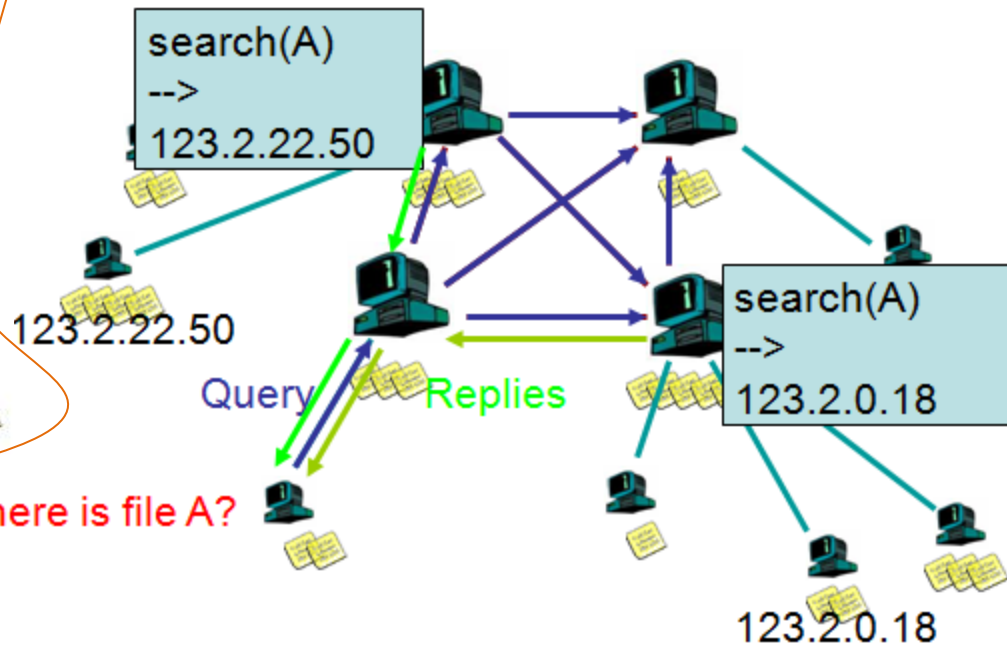


Unstructured Architectures

KaZaA: Inserting Files



KaZaA: Files Search



Where is file A?

Issues:

- Behavior similar to Gnutella, but more efficient
- There is no guarantee on the search time or on the search area

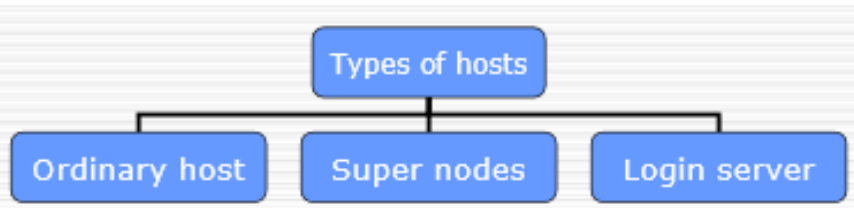
Unstructured Architectures

Partially centralized

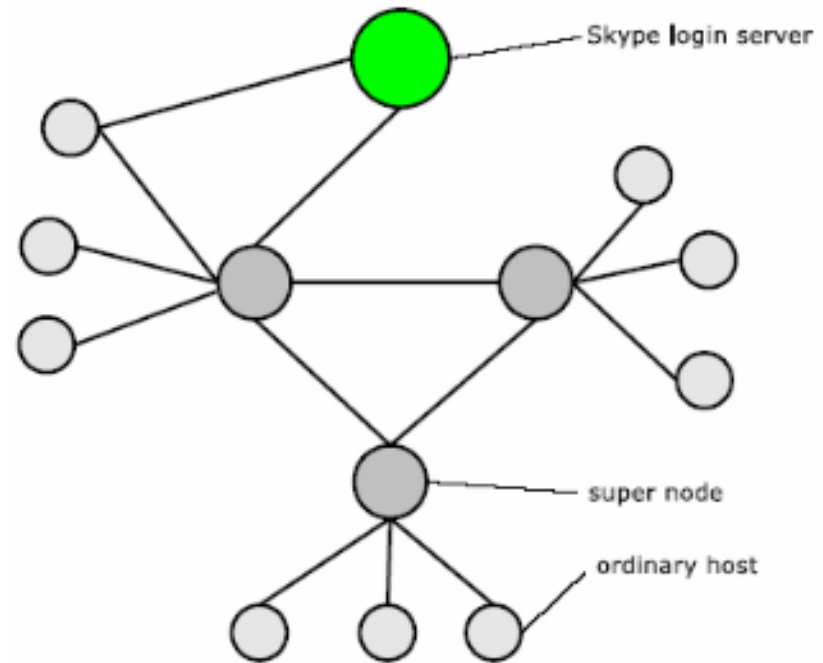
- KaZaA software is a proprietary one
- P2P control data is encrypted
- The messages use HTTP
- A node is either a super-node, or assigned to a super-node
- A super-node has 100-150 child-nodes
- A network can have ~ 30,000 super-nodes
- Each super node has TCP connections with 30-50 super-nodes
- For each file meta-data is maintained (name, size, content hash, file descriptor)
- *The content hash* is used to find another copy of a partially transferred file
- The version without *spyware* and *pop-ups*: **KaZaA-lite**

Unstructured Architectures

Skype



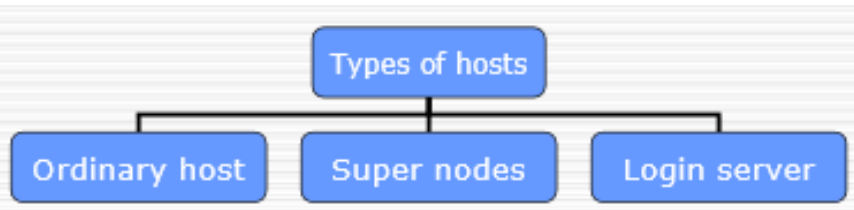
- The first P2P telephony network based on IP
- from June 2014, Microsoft announced the incompatibility with the previous Skype protocol
- Uses Microsoft Notification Protocol 24 (first use -> MSN Messenger in 1999)
- architecture was similar to KaZaA



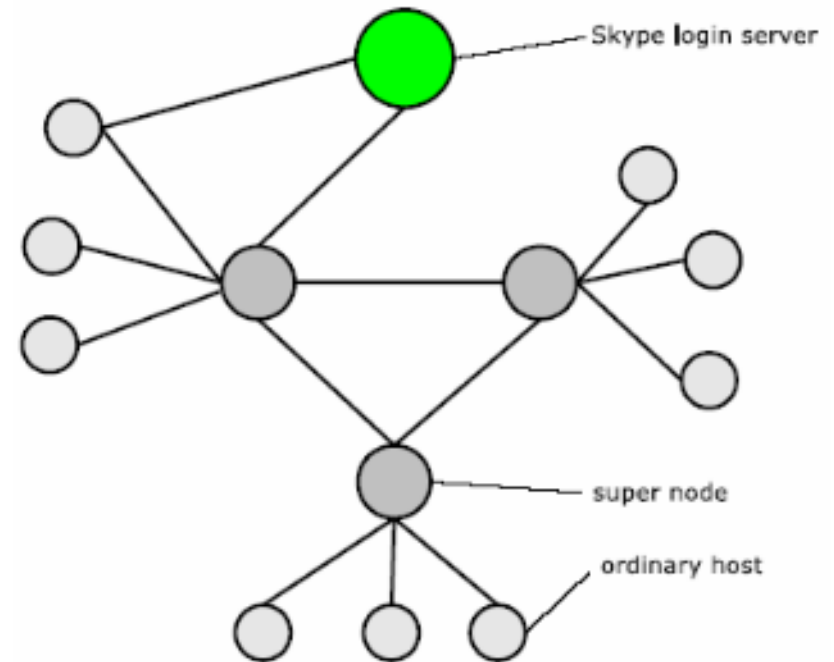
http://www1.cs.columbia.edu/~salman/publications/skype1_4.pdf

Unstructured Architectures

Skype



- Each client maintains a *host cache* with IP addresses and the port numbers associated of the accessible super-nodes
- Any customer with bandwidth (and without firewall or NAT restrictions) could become a super-node
- from 2012, Microsoft began moving super-nodes to the hosting servers in its data centers



<https://en.wikipedia.org/wiki/PRISM>

Unstructured Architectures

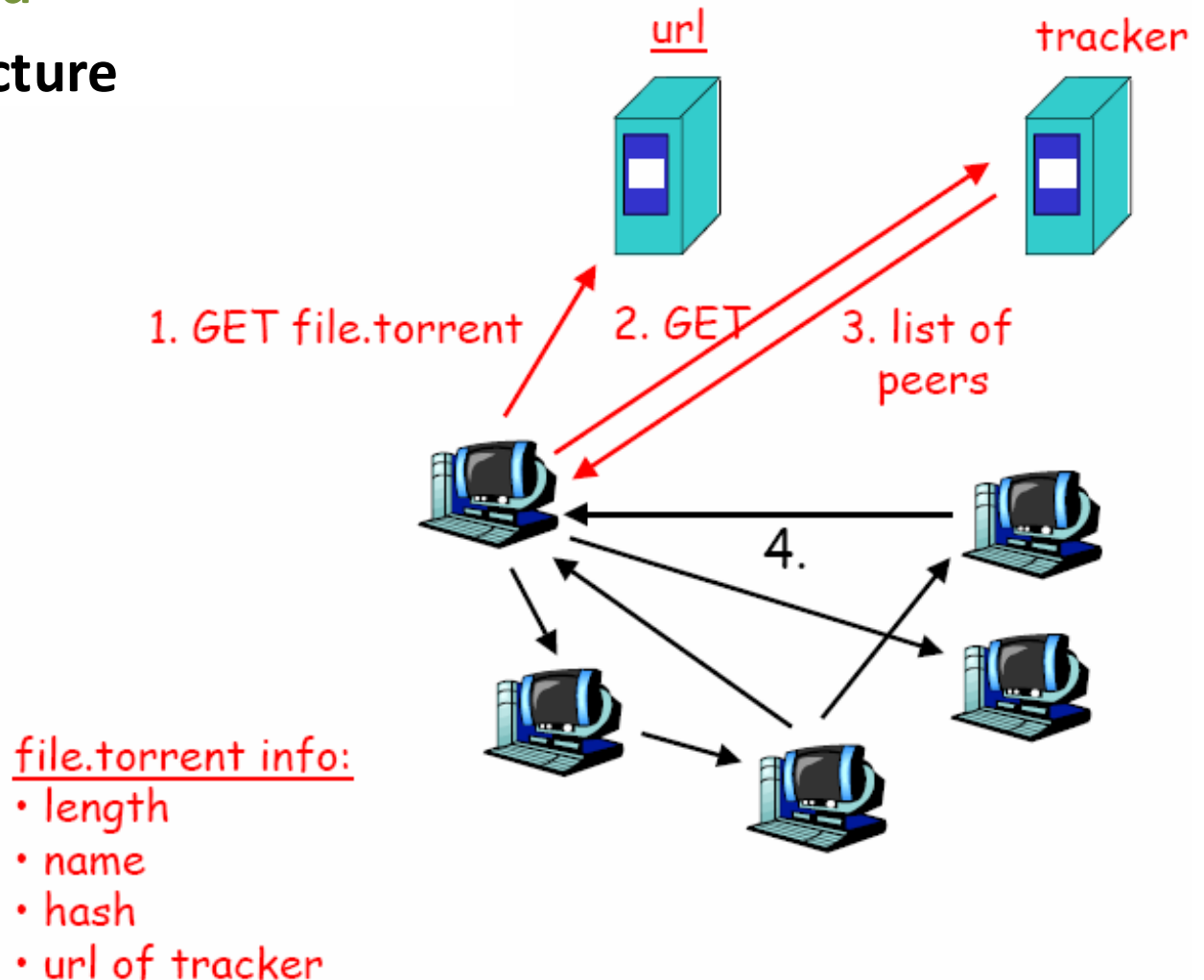
Partially centralized

- If a file is found on multiple nodes, the transfer can be done in parallel
 - The same copies are identified via *content hash*
- Different chunks of the file are transferred from different nodes
- For interrupted transfers, an automatic recovery is performed
- Example: **BitTorrent**
 - In 2002, B. Cohen released BitTorrent
 - The focus was over *efficient fetching* and not on *searching*
 - Supporters since it appeared
 - Blizzard Entertainment used BitTorrent to distribute beta versions of new games

Unstructured Architectures

Partially centralized

BitTorrent - architecture



Unstructured Architectures

Partially centralized

BitTorrent

- It is based on *swarming* mechanism:
 - **Join**: contact a centralized server (*tracker*) and get a list of *peers*
 - **Publish**: running a *tracker* server
 - **Search**: e.g., uses Google to find a *tracker* for the desired file
 - **Fetch**: Take pieces of files from peers;
- Obs.: The difference from Napster
 - *File Chunk Download*
 - Using the “tit-for-tat” strategy: if A *downloads* from other nodes, then A should allow downloads from it (*free-rider problem*)

Unstructured Architectures

Problems

- **Nodes whose IP addresses are available via NAT (with restrictions)**
 - They can not be TCP servers for P2P network
 - Partial solution: *reverse call*
 - A wants to transfer from B and B uses NAT
 - A and B establish TCP connections with C server (which has a routable IP)
 - A may request B, via C, to create a TCP connection from B to A
 - A can send a request to B, and B gives the answer
 - If A and B use NAT?
- **Flash crowd: an unexpected increase of demand for a particular resource**
 - For the wanted content, there are not enough copies
 - How much time does it take for a user to locate the file?
 - How many messages will receive a node due to searches made by other nodes?
 - A generic protocol, based on TTL can be used

Structured Architectures

- Represent academic solutions for P2P
- Scope:
 - Successful search
 - Search time is performed in known boundaries
 - Proven scalability
- Approach: **DHT (Distributed Hash Table)**
 - Pairs (key, value) are stored
 - Key – file names
 - Value – file content or a pointer to a location
 - Each *peer* stores a set (key, value)
 - *Operations: find the node responsible for a Key*
 - Mapping *key – node*
 - Efficient routing to *insert/lookup/delete* operations associated with this node
 - A wide fluctuation of nodes is allowed

Structured Architectures

- Aspects: **content localization**
- Idea: The responsibility is distributed to multiple nodes of the coverage network, in an adaptive manner
- To each resource a unique key is associated via a *hash* function: $h(\text{"Computer Networks Course"}) \rightarrow 7929$; The range values of the *hash function* are distributed in P2P network
- Each node must “know” the location of at least one copy of a resource for which the hash function has values in its range
- Nodes can maintain their own cache with copies of each resource that they need to “know”

Structured Architectures

- Aspects: **routing**
- For each resource, a node that “knows” the resource must be accessed through the shortest path
- Structured P2P systems approaches differ by the routing strategy
- The nodes of the system form a distributed structure that can be: ring, tree, hypercube, etc.
- It provides an API for distributed hash tables (DHT - Distributed Hash Table)
 - Giving a key k , the API will return the IP address of the node responsible for the k key value

Structured Architectures

- **Implementations**

- Chord [MIT]
- Pastry [Microsoft Research UK, Rice University]
- Tapestry [UC Berkeley]
- Content Addressable Network (CAN) [UC Berkeley]
- SkipNet [Microsoft Research US, Univ. of Washington]
- Kademlia [New York University]
- Viceroy [Israel, UC Berkeley]
- P-Grid [EPFL Switzerland]

Structured Architectures

- **Loosely structured**

- The nodes can estimate which nodes store the searched resources
 - Blind broadcasts are avoided
 - *Chain mode propagation* mechanism is used: each node takes local decisions regarding who will be the next node which will be queried
- Search for a file involves using a key and a timeout mechanism
- Example: **Freenet**

[<https://en.wikipedia.org/wiki/Freenet>]

Instruments

- “IPFS is the Distributed Web” - <https://ipfs.io/>
 - A peer-to-peer hypermedia protocol to make the web faster, safer, and more open



HTTP is inefficient and expensive

HTTP downloads a file from a single computer at a time, instead of getting pieces from multiple computers simultaneously. With video delivery, a P2P approach could save 60% in bandwidth costs.

IPFS makes it possible to distribute high volumes of data with high efficiency. And zero duplication means savings in storage.



Humanity's history is deleted daily

The average lifespan of a web page is 100 days. Remember GeoCities? The web doesn't anymore. It's not good enough for the primary medium of our era to be so fragile.

IPFS provides historic versioning (like git) and makes it simple to set up resilient networks for mirroring of data.

Instruments

- “IPFS is the Distributed Web” - <https://ipfs.io/>
 - A peer-to-peer hypermedia protocol to make the web faster, safer, and more open



The web's centralization limits opportunity

The Internet has been one of the great equalizers in human history and a real accelerator of innovation. But the increasing consolidation of control is a threat to that.

IPFS remains true to the original vision of the open and flat web, but delivers the technology which makes that vision a reality.



Our apps are addicted to the backbone

Developing world. Offline. Natural disasters. Intermittent connections. All trivial compared to interplanetary networking. The networks we're using are so 20th Century. We can do better.

IPFS powers the creation of diversely resilient networks which enable persistent availability with or without Internet backbone connectivity.

Instruments

- “IPFS is the Distributed Web” - <https://ipfs.io/>

Let's take a look at what happens when you add files to IPFS:



Each file and all of the **blocks within it** are given a **unique fingerprint** called a **cryptographic hash**.



IPFS **removes duplications** across the network and tracks **version history** for every file.



Each **network node** stores only content it is interested in, and some indexing information that helps figure out who is storing what.



When **looking up files**, you're asking the network to find nodes storing the content behind a unique hash.



Every file can be found by **human-readable names** using a decentralized naming system called **IPNS**.

- <https://github.com/ipfs/papers/raw/master/ipfs-cap2pfs/ipfs-p2p-file-system.pdf>

Summary

- **Peer-to-peer (P2P) paradigm**
 - Preliminaries
 - Definitions
 - Characteristics
 - Application types
 - Infrastructures
 - Instruments

Bibliography

- P2P Networking and Applications, John F. Buford, Heather Yu, Eng Keong Lua, 2009, Elsevier
- <https://www.kirsle.net/skype-switched-to-the-msn-messenger-protocol>



Questions?

Questions?